FAISS Retrieval Pipeline Execution Guide

This document outlines the execution order and purpose of each script in the FAISS benchmark pipeline for the 532 systems project.

1. Environment Setup (Virtual Environment)

Commands (Windows):

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
```

Commands (Mac/Linux):

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

Description:
Creates an isolated Python virtual environment, activates it, and installs all the necessary dependencies required to run the pipeline safely without conflicting with system packages.

(Note: Before proceeding to the next step, ensure you have manually created a `data/` folder in the root directory and placed the raw BEIR `trec-covid` dataset inside it.)

2. Generate the Dataset

Command:

```
python verify_and_test_pipeline.py
```

Description:
Downloads the raw BEIR trec-covid dataset, slices it to 100,000 passages, and generates 384-dimensional embeddings using the `all-MiniLM-L6-v2` model. It creates synthetic

metadata (category, access group, timestamp, selectivity buckets) and computes exact brute-force top-10 ground truth answers for recall verification. All artifacts are saved to `data/processed/`. *(Expect this to take over an hour to complete.)*

3. Run the Filter Selectivity Benchmark

Command:

```
python evaluate_faiss.py
```

Description:
Loads the FAISS index and runs single-threaded vector searches to evaluate metadata filter impact. It measures Queries Per Second (QPS), p99 latency, RAM footprint, and CPU usage across four filter selectivities: unfiltered, 50%, 10%, and 1%. Metrics are exported to `metrics/faiss_benchmark_results.csv`.

4. Run the Concurrency Stress Test

Command:

```
python evaluate_faiss_concurrency.py
```

Description:
Uses multi-threading to simulate 1, 4, 8, 16, and 32 simultaneous clients querying the FAISS index. This test isolates vector search contention by tracking how throughput drops and p99 latency spikes when the CPU hits 100% saturation. Metrics are exported to `metrics/faiss_concurrency_results.csv`.

5. Generate Performance Graphs

Command:

```
python generate_faiss_graphs.py
```

Description:
Reads the CSV metric files and utilizes `matplotlib` to generate four visualization charts

(QPS & p99 latency by selectivity, and QPS & p99 latency by concurrency). The PNG charts are saved directly into the `graphs/` directory.

6. Export Dependencies

Command:

```
pip freeze > requirements.txt
```

Description:

Exports all active Python virtual environment packages and versions so the exact runtime environment can be replicated by the rest of the team. (Only necessary when adding new packages to the project).